

Producer–Consumer using Anonymous Pipe

Question

Write a C program to implement communication between a producer and consumer using an anonymous pipe. The producer should send data through the pipe and the consumer should receive and display the data.

warranty, not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```
saiKrishna_90156@DESKTOP-2JF9H5T:~$ gcc -Wall -Wextra Practical5.c -o Practical5
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5.c
-bash: ./Practical5.c: Permission denied
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5.c
-bash: ./Practical5.c: Permission denied
saiKrishna_90156@DESKTOP-2JF9H5T:~$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5.c
-bash: ./Practical5.c: Permission denied
saiKrishna_90156@DESKTOP-2JF9H5T:~$ chmod +x Practical5
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5.c
-bash: ./Practical5.c: Permission denied
saiKrishna_90156@DESKTOP-2JF9H5T:~$ gcc -Wall -Wextra Practical5.c -o Practical5
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5
total 20
-rwxr-xr-x 1 saiKrishna_90156 saiKrishna_90156 16344 Sep 27 16:51 Practical5
-rw-r--r-- 1 saiKrishna_90156 saiKrishna_90156 1152 Sep 27 16:36 Practical5.c
Command execution completed.
saiKrishna_90156@DESKTOP-2JF9H5T:~$ gcc Practical5.c -o Practical5
saiKrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical5
total 20
-rwxr-xr-x 1 saiKrishna_90156 saiKrishna_90156 16344 Sep 27 16:51 Practical5
-rw-r--r-- 1 saiKrishna_90156 saiKrishna_90156 1152 Sep 27 16:36 Practical5.c
Command execution completed.
saiKrishna_90156@DESKTOP-2JF9H5T:~$ cat Practical5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {

    int fd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    // First child: ls -l
    pid1 = fork();
```

```

pid1 = fork();

if (pid1 < 0) {
    perror("fork");
    return 1;
}

if (pid1 == 0) {
    // Redirect stdout to pipe
    dup2(fd[1], STDOUT_FILENO);

    close(fd[0]);
    close(fd[1]);

    execlp("ls", "ls", "-l", NULL);

    perror("exec ls");
    exit(1);
}

// Second child: grep "."
pid2 = fork();

if (pid2 < 0) {
    perror("fork");
    return 1;
}

if (pid2 == 0) {
    // Redirect stdin from pipe
    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);
    close(fd[1]);

    execlp("grep", "grep", ".", NULL);

    perror("exec grep");
    exit(1);
}

// Parent closes both pipe ends
close(fd[0]);
close(fd[1]);

// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Command execution completed.\n");

```